

# AgentBroko Complete User Journey & Usage Guide

This guide breaks down what users experience, how they use AgentBroko, and the exact deliverables they get at every stage.

## ■ What Users Get from AgentBroko

- **Zero Cloud Dependencies & 100% Privacy:** All video rendering, speech generation, and PDF processing run on the user's local machine with **0 KB network calls and no API keys required**.
- **Coding-Agent Friendly:** Structured CLI designed for humans and autonomous coding assistants (Cursor, Cline, Roo, Antigravity).
- **Dual Packaging:** Can be run globally with npm / npx or installed into Python environments via pip.

## ■■■■ User Journey 1: Installation & Discovery

**Workflow:** User visits [agentbroko.vercel.app](https://agentbroko.vercel.app) → Copies `npm install -g agentbroko` → Runs `npx agentbroko skills` → Discovers Video Forge, PDF Playbook & PDF Tools

### Step 1: Running without installation (Instant Trial)

```
$ npx agentbroko skills
```

### Step 2: Installing globally

```
$ npm install -g agentbroko
```

Output the user sees:

```
AgentBroko skills
video-forge Local video editing, offline narration, and captions
pdf Local PDF inspection, text extraction, and page rendering
pdf-playbook Premium branded developer handbook PDF generation
Use: agentbroko [options]
```

## ■ User Journey 2: Creating a Video with Video Forge

**Pipeline:** 1. doctor → 2. init my-video → 3. Edit script.txt & project.json → 4. speak (Offline TTS → .wav) → 5. captions (Auto SRT → .srt) → 6. render (FFmpeg → final.mp4)

### Step 1: Health & Environment Check

The user checks if their system has the necessary binaries (FFmpeg, TTS engine):

```
$ agentbroko video-forge doctor
```

User Output:

```
FFmpeg: found
FFprobe: found
Offline TTS: windows
```

## Step 2: Scaffold a Video Project

```
$ agentbroko video-forge init my-video
```

What gets created in the user's filesystem:

```
my-video/  
■■■ project.json # Structured video timeline & clip coordinates  
■■■ script.txt # Spoken text narration  
■■■ audio/ # Target folder for generated voiceovers  
■■■ captions/ # Target folder for generated SRT subtitles  
■■■ media/ # Folder for input clips/images  
■■■ outputs/ # Folder for the final rendered MP4
```

## Step 3: Generate Spoken Narration (Offline TTS)

The user writes their narration in my-video/script.txt (or passes text directly) and generates a clean .wav audio track:

```
$ agentbroko video-forge speak --file my-video/script.txt --output my-video/audio/narration.wav
```

User Output:

```
Created my-video/audio/narration.wav with windows (or piper)
```

## Step 4: Generate Synchronized SRT Subtitles

The user generates timed subtitles from their script:

```
$ agentbroko video-forge captions --file my-video/script.txt --output my-video/captions/subtitles.srt
```

User Output:

```
Created my-video/captions/subtitles.srt
```

## Step 5: Validate and Render the Final MP4 Video

```
# Validate project schema  
$ agentbroko video-forge validate my-video/project.json  
$  
# Render final video  
$ agentbroko video-forge render my-video/project.json
```

**User Deliverable:** A broadcast-quality outputs/final.mp4 video with synchronized speech narration, background music ducking, and styled subtitles.

## ■ User Journey 3: Creating a 20-Page Developer Handbook (PDF Playbook)

**Pipeline:** agentbroko pdf-playbook → Interactive Wizard (Title, Audience, Promise) → ReportLab Document Engine → Outputs 20-Page Dark-Theme Handbook (.pdf)

### Option A: Interactive Wizard Mode

```
$ agentbroko pdf-playbook --output my-playbook.pdf
```

- The CLI prompts: *What is your guide title?* (e.g. AI Agent Engineering Guide)
- The CLI prompts: *Who is the target audience?* (e.g. Full-Stack Developers)
- The CLI prompts: *What is the core promise?* (e.g. Master local AI model routing)

### Option B: Automated / CI/CD Mode

```
$ agentbroko pdf-playbook --non-interactive --output production-playbook.pdf
```

**User Deliverable:** A 34 KB, publication-ready, dark-theme 20-page A4 PDF complete with table of contents, structured tables, and checklists.

## ■ User Journey 4: Local Offline PDF Utilities

```
# 1. Inspect metadata (page count, encrypted status, author)
$ agentbroko pdf info document.pdf
$
# 2. Extract clean text to file (zero cloud upload)
$ agentbroko pdf text document.pdf --output document.txt
$
# 3. Render all pages as high-res images (requires Poppler)
$ agentbroko pdf render document.pdf --output rendered-pages/
```

## ■ User Journey 5: How Coding Agents (Cursor, Cline, Roo) Use AgentBroko

Coding agents invoke AgentBroko as a subprocess tool:

- **Determinism:** Every command returns explicit POSIX exit codes (0 for success, non-zero for validation/runtime errors).
- **No Prompts in Headless Mode:** Commands like `--non-interactive` allow autonomous agents to build media without human intervention.
- **Structured JSON:** Agents read and modify `project.json` directly to adjust video timelines.

## ■ Summary of Deliverables by Command

| Command                            | User Input         | Output Generated        | Engine               |
|------------------------------------|--------------------|-------------------------|----------------------|
| agentbroko skills                  | None               | Skills catalog list     | CLI Hub              |
| agentbroko video-forge init <name> | Project Name       | Project folder scaffold | Template             |
| agentbroko video-forge speak       | Text / Script file | .wav speech audio       | Windows SAPI / Piper |
| agentbroko video-forge captions    | Text script        | .srt subtitle file      | Subtitle Lexer       |
| agentbroko video-forge render      | project.json       | final.mp4 video file    | FFmpeg               |
| agentbroko pdf-playbook            | Title & Audience   | 20-page .pdf handbook   | ReportLab            |
| agentbroko pdf text                | Input .pdf         | Clean text .txt file    | pypdf                |